Investigation of Bike-Sharing Demand Patterns, Fleet Allocation, and Hourly Demand Estimation

Empirical Analysis and Decision-Support Prototype on the Capital Bikeshare Dataset (2011–2012)

Submitted by: AMAN KUMAR RAJ | **Roll No.:** 23CEB0A27

Dataset: UCI Machine Learning Repository — Bike Sharing Dataset (17,379 Hourly & 731 Daily Observations)

1. Introduction and Problem Statement

Urban bike-sharing networks frequently encounter an operational imbalance in which riders report empty docks at high-demand stations while a substantial number of bicycles remain unutilized elsewhere in the city. The objective of this study is to examine two years of historical operational data from the Capital Bikeshare system in Washington, D.C. (2011–2012) and determine whether addressing rider complaints primarily requires acquiring additional bicycles or improving fleet reallocation and shift planning.

To answer this question rigorously, this work is organized around four core tasks: (1) systematic data quality auditing and leakage-free feature engineering on the raw hourly and daily logs; (2) pre-registered statistical hypothesis testing on commuting-window demand; (3) development and chronological validation of an hourly demand-estimation model; and (4) implementation of an interactive Streamlit prototype equipped with a natural-language query engine that allows operations managers to inspect demand profiles and test scheduling scenarios.

1.1 Summary of Principal Findings

Across the 731-day study period (`3,292,679` total checkouts over `17,379` recorded hours), the overall mean demand is only `189.5` rentals per hour (median `142.0` rentals/hour), whereas the 99th-percentile hourly demand reaches `782` to `977` rentals per hour. On working days (`n = 500` days), **51.3% of all daily checkouts occur within just five commuting hours** (`07:00–08:59` and `16:00–18:59`), peaking at `477.0` rentals/hour at `08:00` (90th percentile `692`) and `525.3` rentals/hour at `17:00` (90th percentile `782`). Between these peaks, demand falls by more than 60% during midday (`10:00–15:00`) and by more than 95% overnight (`00:00–05:00`). On weekends and holidays (`n = 231` days), the morning commuter spike disappears (`105.7` rentals/hour at `08:00`) and ridership shifts to a single midday recreational plateau between `12:00` and `16:00` averaging `359–373` rentals/hour.

Consequently, **simultaneous rider complaints and idle bicycles are primarily a temporal reallocation problem rather than a 24-hour fleet deficit**. Purchasing additional bicycles without modifying truck-rebalancing schedules would leave more inventory idle during the 19 off-peak hours of each working day. At the same time, total annual ridership increased by **64.9%** from `1,243,103` rentals in 2011 to `2,049,576` rentals in 2012 (with registered subscribers generating `81.2%` of rides). Thus, while daily stockouts should be addressed immediately through pre-peak staging before `07:00` and inter-peak rebalancing between `10:00` and `15:30`, medium-term capacity expansion at major commuter corridors is warranted to keep pace with secular system growth.

2. Audit of Prior Time-Series Report and Methodological Corrections

Prior to building the Python pipeline, I audited the existing R Markdown report (`time-series-analysis.html`) included in the workspace. Although that reference report provided a useful exploratory overview of daily rentals, it contained five methodological limitations that prevented its direct use for operational forecasting:

1. **Preprocessing Data Leakage Across Train and Test Sets:** The legacy script applied outlier cleaning (`forecast::tsclean()`), centered moving averages (`ma(order = 7)` and `ma(order = 30)`), and seasonal-trend decomposition (`stl()`) to the entire 731-day series prior to slicing the holdout window (indices `700:725`). Centered windows at day t incorporate future observations $t + 1, \ldots, t + k$, leaking holdout information into the training series.
2. **Omission of Hourly Granularity:** The legacy report analyzed only `day.csv` (731 rows) and ignored `hour.csv` (17,379 rows). Daily aggregation conceals the morning and evening rush-hour spikes that drive dock stockouts.
3. **Removal of Genuine Extreme Events:** Applying `tsclean()` prior to evaluation replaced real operational shocks—such as the Hurricane Sandy shutdown on 29 October 2012—with smoothed interpolations, distorting holdout error metrics.
4. **Misspecified Seasonal Period:** Daily series were defined using `ts(..., frequency = 30)`, imposing an artificial 30-day cycle rather than the true 7-day weekly and 365.25-day annual periodicities.
5. **Absence of Weather and Calendar Predictors:** The univariate `auto.arima(1,1,1)` specification ignored temperature, precipitation, humidity, wind speed, and working-day indicators.

In the present implementation, the original HTML file is preserved intact for reference, and a clean modular Python pipeline (`src/`) is constructed from scratch with strict chronological splitting and zero target leakage.

3. Data Quality Verification and Feature Engineering

3.1 Automated Data Quality Checks

The raw UCI files (`data/raw/hour.csv` and `data/raw/day.csv`) were validated using 26 programmatic checks in `src/data_validation.py`. Table 1 summarizes the principal audit findings.

Table 1. Summary of automated data quality checks performed on `hour.csv` and `day.csv`.

Check Category	Verification Rule	Observed Result	Analytical Treatment
Dimensions & Schema	Match UCI specification (17 hourly cols, 16 daily cols)	`hour.csv`: 17,379 × 17 `day.csv`: 731 × 16	Exact match verified; raw files kept read-only.
Missing & Duplicate Rows	Check NaN cells, duplicate rows, duplicate `instant`, and duplicate `(dteday, hr)`	0 missing values, 0 duplicates, strictly monotonic timestamps	No records dropped or altered.

Check Category	Verification Rule	Observed Result	Analytical Treatment
Component Additivity	Verify `casual + registered == cnt` and non-negativity (`cnt ≥ 0`)	0 mismatches across all 17,379 hourly and 731 daily records	`casual` and `registered` excluded from predictors to prevent target leakage.
Cross-File Consistency	Aggregate `hour.csv` by `dteday` and compare sums against `day.csv`	0 dates disagree (maximum absolute deviation = 0 rentals)	Confirmed `day.csv` is the exact daily sum of `hour.csv`.
Hourly Sequence Gaps	Compare 17,379 rows against full `731 × 24 = 17,544` calendar grid	165 absent hours across 76 dates (655 complete 24-hr days; 62 with 23 hrs; 14 with ≤22 hrs)	Documented, not imputed with synthetic zeros; largest gap is 23 missing hours on 2012-10-29 (Hurricane Sandy).
Season Code Audit	Inspect calendar dates of `season` transitions (codes 1..4)	Transitions occur on equinox/solstice dates (~Mar 21, Jun 21, Sep 23, Dec 21)	Code 1 (labeled `springer` in README) corresponds to Dec 21–Mar 20; documented in UI and reports.
Weather Sensor Audit	Check physical bounds on `temp, atemp, hum, windspeed ∈ [0, 1]`	22 hours on `2011-03-10` record `hum = 0.0` during light rain; 24 hours on `2012-08-17` show constant `atemp = 0.2424`	Flagged as sensor anomalies in quality report; raw values retained for reproducibility.
Distributional Outliers	Global IQR rule (`cnt > 642.5`) and conditional robust z-score (`|z_robust| > 3.5`)	505 global IQR outliers; 438 conditional outliers within `hr × workingday`	Retained in dataset because high values represent genuine rush-hour demand peaks.

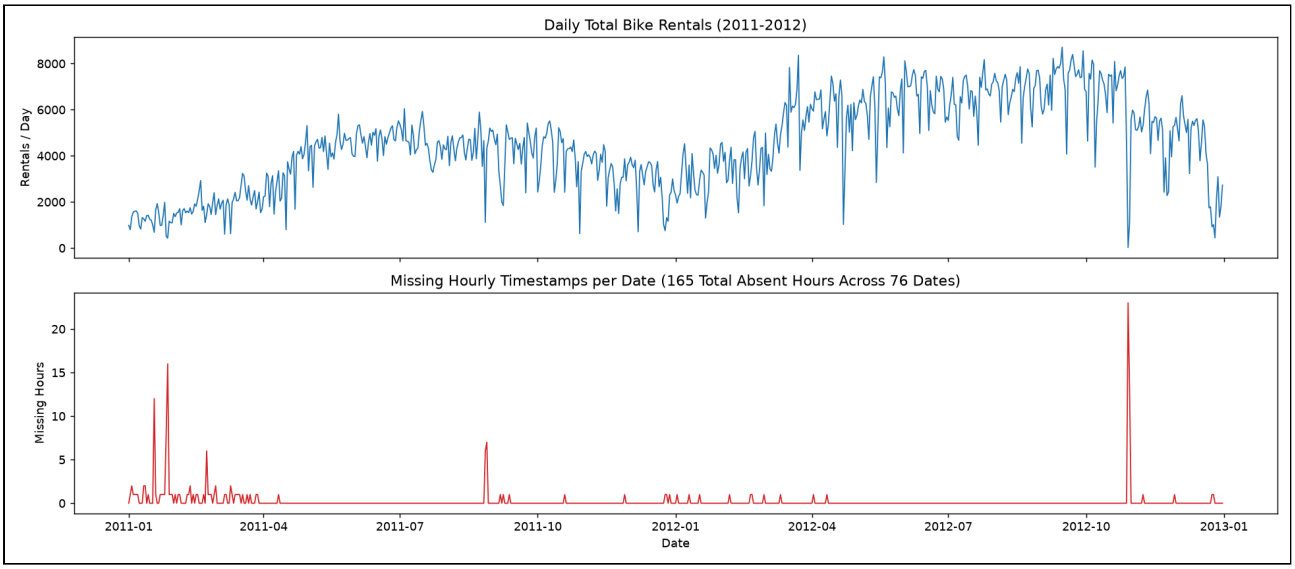


Figure 1. Daily total bike rentals across 2011–2012 (top) and distribution of the 165 missing hourly timestamps across 76 calendar dates (bottom).

3.2 Feature Engineering and Leakage Prevention

All feature transformations are implemented in `src/features.py` as deterministic row-wise functions of calendar and weather variables so that no future or target information leaks across observations. First, the normalized weather measurements are rescaled to physical units for interpretability:

$$T_C = 41 \cdot \texttt{temp}, \quad AT_C = 50 \cdot \texttt{atemp}, \quad H_{\%} = 100 \cdot \texttt{hum}, \quad W_{km/h} = 67 \cdot \texttt{windspeed} \tag{1}$$

Second, to preserve periodic continuity across midnight (hour 23 to hour 0), week boundaries, and year-end transitions, cyclical Fourier encodings are constructed for hour of day, day of week, and month:

$$\texttt{hr_sin} = \sin(2\pi \cdot \texttt{hr} / 24), \quad \texttt{hr_cos} = \cos(2\pi \cdot \texttt{hr} / 24) \tag{2}$$

$$\texttt{weekday_sin} = \sin(2\pi \cdot \texttt{weekday} / 7), \quad \texttt{weekday_cos} = \cos(2\pi \cdot \texttt{weekday} / 7) \tag{3}$$

$$\texttt{mnth_sin} = \sin(2\pi \cdot (\texttt{mnth} - 1) / 12), \quad \texttt{mnth_cos} = \cos(2\pi \cdot (\texttt{mnth} - 1) / 12) \tag{4}$$

Third, domain-specific indicators are defined for weekends (`is_weekend = 1` if `weekday ∈ {0, 6}`), commuting windows (`is_commute_hour = 1` if `hr ∈ {7, 8, 16, 17, 18}`), and their interaction with working-day status (`workday_commute = is_commute_hour × workingday`). Because `cnt = casual + registered` holds identically for every record, including either component count as a predictor would introduce target leakage. The helper function `assert_no_leakage()` enforces at runtime that none of `(casual, registered, cnt, instant)` can be included in the model feature matrix.

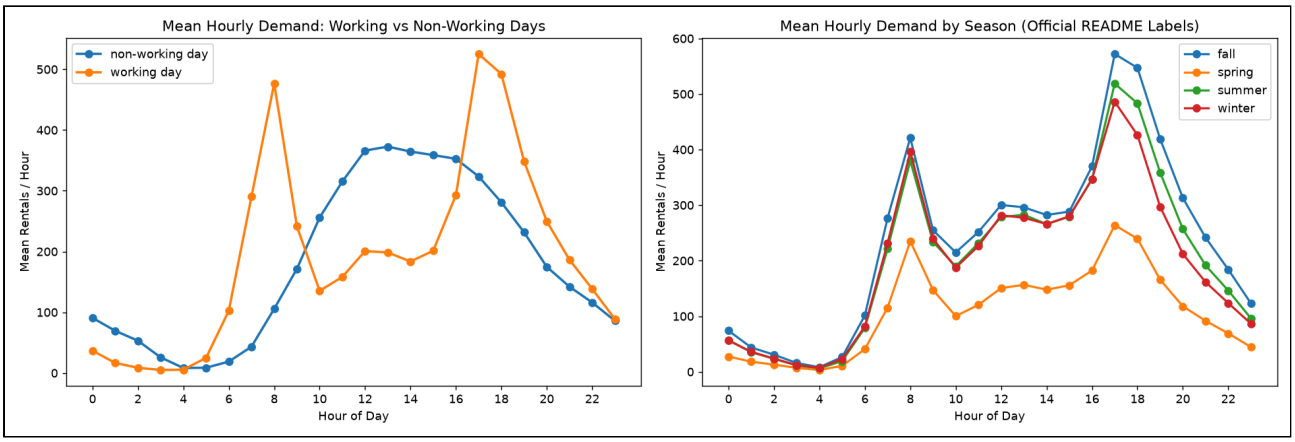


Figure 2. Mean hourly rental profiles comparing working vs. non-working days (left) and across the four dataset season codes (right).

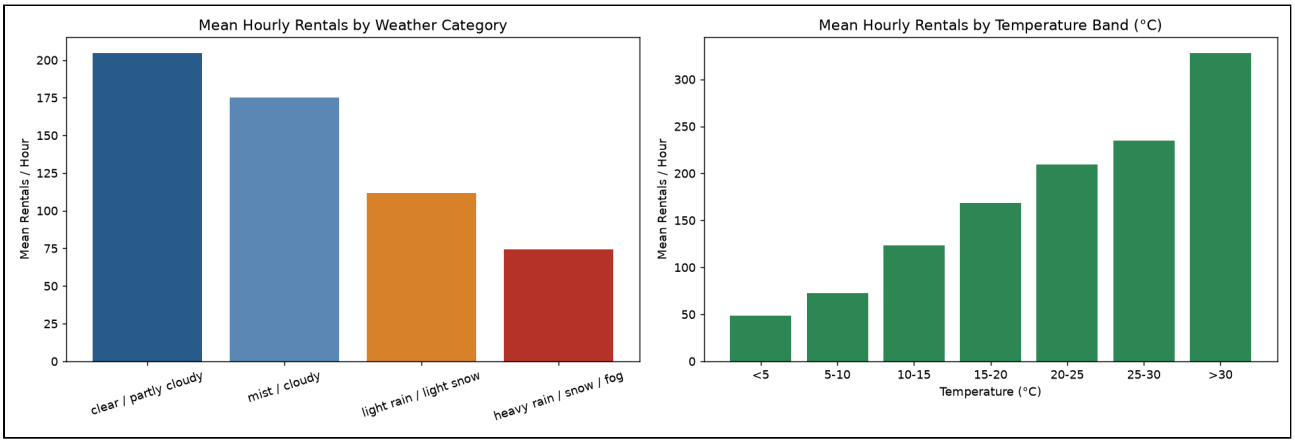


Figure 3. Mean hourly bike rentals across weather situation categories (left) and ambient temperature bands in degrees Celsius (right).

4. Hypothesis Formulation and Statistical Testing

4.1 Experimental Design and Hypotheses

To test whether commuting-window demand is structurally higher on working days than on weekends and holidays, the commuting windows were fixed prior to testing as Morning (`07:00–08:59`, `hr ∈ {7, 8}`) and Evening (`16:00–18:59`, `hr ∈ {16, 17, 18}`). Because hourly observations within the same day are correlated, the unit of analysis is defined as a single calendar date d (N = 731 days), with daily commuting-window volume given by:

$$Y_d = \Sigma_{h \in \{7,8,16,17,18\}} \texttt{cnt}_{d,h} \tag{5}$$

Let μ_1 and μ_0 denote the population mean commuting-window rental totals on working days (`workingday = 1`, n_1 = 500) and non-working days (`workingday = 0`, n_0 = 231), respectively. We test at significance level α = 0.05:

- **Null Hypothesis (H_0):** $\mu_1 = \mu_0$ (mean daily commuting-window demand is equal on working and non-working days).
- **Alternative Hypothesis (H_1, one-sided):** $\mu_1 > \mu_0$ (mean daily commuting-window demand is higher on working days).

Because Levene's test indicates unequal group variances ($p = 6.34 \times 10^{-10}$; SD_1 = 862.9 vs. SD_0 = 566.4), the primary parametric test is Welch's unequal-variance t-test with Satterthwaite degrees of freedom ν, supplemented by the standardized effect size (Hedges' g) and the nonparametric Mann-Whitney U test:

$$t = (\bar{Y}_1 - \bar{Y}_0) / \sqrt{(s_1^2/n_1 + s_0^2/n_0)}, \quad g = J(\nu) \cdot (\bar{Y}_1 - \bar{Y}_0) / s_{pooled} \tag{6}$$

Furthermore, because daily residuals exhibit positive serial correlation across consecutive days (lag-1 autocorrelation r_1 = 0.79, Durbin-Watson = 0.41) and weather varies across the year, I also estimate a semi-logarithmic Ordinary Least Squares (OLS) regression with Newey-West heteroskedasticity- and autocorrelation-consistent (HAC) standard errors (7-day lag bandwidth):

$$\log(1 + Y_d) = \beta_0 + \beta_1 \texttt{workingday}_d + \beta_2 \texttt{temp}_d + \beta_3 \texttt{temp}_d^2 + \beta_4 \texttt{hum}_d + \beta_5 \texttt{windspeed}_d + \gamma_{weather} + \delta_{month} + \theta_{yr_d} + \epsilon_d \tag{7}$$

4.2 Empirical Results

Table 2 reports the test statistics across the full 731-day sample and four sensitivity subsets. On working days, commuting-window rentals average `2,068.5` rides/day (median `2,078.0`) compared to `1,105.2` rides/day (median `1,096.0`) on non-working days—an increase of `+963.3` rides/day (95% Welch CI: `[858.0, 1,068.7]`; 10,000-draw bootstrap 95% CI: `[857.3, 1,069.2]`). Welch's test yields t = 17.96 (df = 645.6, one-sided $p = 4.33 \times 10^{-59}$) with Hedges' g = 1.23 (very large effect size). The Mann-Whitney test confirms this finding (U = 94,178.5, one-sided $p = 3.65 \times 10^{-43}$, rank-biserial r = 0.631).

Table 2. Commuting-window hypothesis test results across primary and sensitivity samples.

Sample / Subset	n_{work} / $n_{non\text{-}work}$	Working Mean (Median)	Non-Working Mean (Median)	Mean Diff. (95% CI)	Welch t (df)	One-Sided p	Hedges' g
Primary (All 731 Days)	500 / 231	2,068.5 (2,078.0)	1,105.2 (1,096.0)	+963.3 (858.0, 1,068.7)	17.96 (645.6)	4.33×10^{-59}	1.23
Complete 5-Hr Windows Only	495 / 230	2,087.3 (2,093.0)	1,109.4 (1,100.0)	+977.9 (873.4, 1,082.4)	18.38 (636.2)	3.89×10^{-61}	1.27
Year 2011 Only (`yr = 0`)	250 / 115	1,535.6 (1,739.0)	858.6 (880.0)	+677.0 (570.6, 783.4)	12.52 (302.6)	1.48×10^{-29}	1.24
Year 2012 Only (`yr = 1`)	250 / 116	2,601.4 (2,702.0)	1,349.6 (1,471.0)	+1,251.8 (1,108.8, 1,394.8)	17.23 (282.1)	2.87×10^{-46}	1.77
Weekends Only (Excl. Holidays)	500 / 210	2,068.5 (2,078.0)	1,107.0 (1,100.0)	+961.5 (855.2, 1,067.8)	17.76 (597.1)	2.72×10^{-57}	1.23

In the Newey-West HAC regression (Equation 7, $R^2 = 0.682$), the working-day coefficient is $\hat{\beta}_1 = +0.6372$ (HAC $z = 19.50$, one-sided $p = 5.85 \times 10^{-85}$), indicating that working days generate exp(0.6372) = **1.89×** (95% CI: `[1.77×, 2.02×]`) the commuting-window demand of non-working days after holding weather, month, and year constant. A secondary test comparing rainy/snowy days (at least one hour with `weathersit ≥ 3`, $n = 300$, mean `4,014.2` rentals/day) against dry/mist days ($n = 431$, mean `4,845.5` rentals/day) confirms that precipitation significantly depresses daily demand (Welch $t = 5.89$, $p = 3.02 \times 10^{-9}$, stratified reduction = `-828.7` rentals/day).

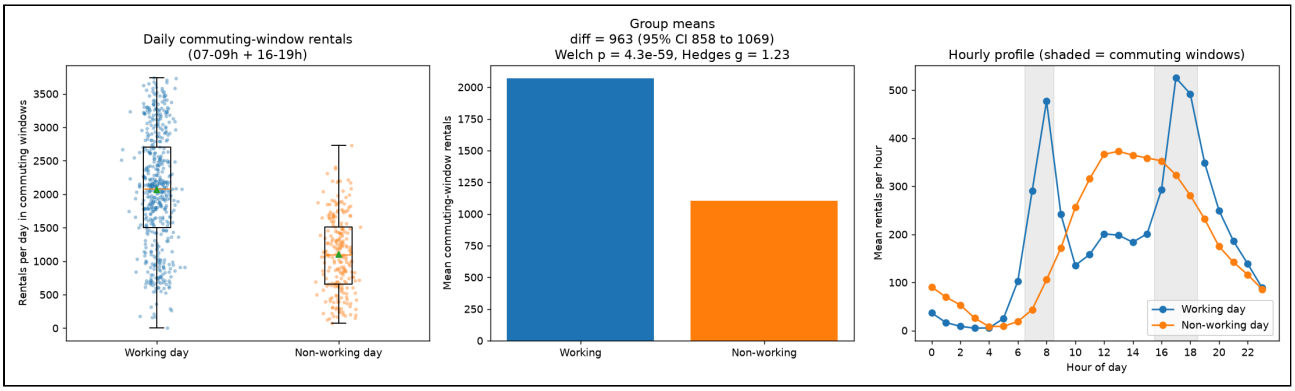


Figure 4. Comparison of daily commuting-window rentals (07:00–08:59 and 16:00–18:59) between working days (n = 500) and non-working days (n = 231).

5. Hourly Demand-Estimation Model

5.1 Chronological Validation Strategy and Model Formulation

Because hourly bike demand is chronologically ordered and exhibits strong autocorrelation, random train/test splitting would leak contemporaneous weather and demand conditions across adjacent hours. I therefore

partitioned `hour.csv` chronologically into an 18-month development set (`2011-01-01` through `2012-06-30`, `13,003` hours) and a 6-month held-out test set (`2012-07-01` through `2012-12-31`, `4,376` hours). Within the development period, hyperparameters were tuned using 4-fold expanding-window cross-validation (validating sequentially on Q3 2011, Q4 2011, Q1 2012, and Q2 2012).

Including the first six months of 2012 in the development window is necessary because total ridership expanded by 64.9% between 2011 (`yr = 0`) and 2012 (`yr = 1`). A diagnostic experiment trained solely on 2011 data confirmed that when `yr = 0` is constant during training, tree-based models cannot learn the 2012 level shift and incur a systematic mean bias of `−89.65` rentals/hour on 2012 data (even though gradient boosting still improves R^2 from `0.435` to `0.623` over the 2011 baseline).

Because hourly rental counts are right-skewed and non-negative, the primary model (`hgb_log`) fits a histogram-based gradient boosting regressor on log-transformed counts and applies a non-negative inverse transformation:

$$z_{d,h} = \log(1 + \mathrm{cnt}_{d,h}), \quad \hat{y}_{d,h} = \max(0,\ \exp(\hat{z}_{d,h}) - 1) \tag{8}$$

Model accuracy on the held-out test set is evaluated using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and the coefficient of determination (R^2):

$$\mathrm{MAE} = (1 / N)\ \Sigma_{i=1..N}\ |y_i - \hat{y}_i|, \quad \mathrm{RMSE} = \sqrt{[\ (1 / N)\ \Sigma_{i=1..N}\ (y_i - \hat{y}_i)^2\ ]}, \quad R^2 = 1 - [\ \Sigma(y_i - \hat{y}_i)^2\ /\ \Sigma(y_i - \bar{y})^2\ ] \tag{9}$$

To quantify forecast uncertainty, out-of-fold development residuals ($r_i = y_i - \hat{y}_i$) are grouped into five predicted-demand quintiles, and the 10th and 90th empirical percentiles ($q_{0.10}$, $q_{0.90}$) are added to point predictions to form a non-negative 80% prediction interval:

$$[\hat{L}_{d,h},\ \hat{U}_{d,h}] = [\ \max(0,\ \hat{y}_{d,h} + q_{0.10}(r_{\mathrm{bucket}})),\ \ \max(0,\ \hat{y}_{d,h} + q_{0.90}(r_{\mathrm{bucket}}))\ ] \tag{10}$$

5.2 Model Evaluation Results

Table 3 compares the historical conditional baseline (mean rentals by `hr × workingday`), a log-target Random Forest (`rf_log`), a Poisson-loss HistGradientBoosting model (`hgb_poisson`), and the selected log-target HistGradientBoosting model (`hgb_log`). On the held-out July–December 2012 test period (4,376 hours, mean actual demand = `254.1` rentals/hour), `hgb_log` achieves **MAE = 43.54 rentals/hour**, **RMSE = 68.18 rentals/hour**, and **R^2 = 0.904**, reducing test MAE by **59.4%** and RMSE by **54.7%** relative to the baseline model. The empirical 80% prediction intervals cover **79.8%** of actual held-out test observations.

Table 3. Expanding-window cross-validation and held-out test performance across candidate models.

Model Specification	Selected Hyperparameters (Dev CV)	Dev CV MAE	Dev CV RMSE	Test MAE	Test RMSE	Test R^2	Test Bias
Baseline (`hr × workingday` mean)	Conditioned historical lookup table	73.46	105.22	107.21	150.35	0.532	−87.26
Random Forest (`rf_log`)	`n_estimators=200, max_features=1.0, min_samples_leaf=1`	52.80	77.96	—	—	—	—

Model Specification	Selected Hyperparameters (Dev CV)	Dev CV MAE	Dev CV RMSE	Test MAE	Test RMSE	Test R^2	Test Bias
HistGradientBoosting (`hgb_poisson`)	`loss=poisson, lr=0.05, max_leaf_nodes=31, min_leaf=50`	46.09	69.64	—	—	—	—
Selected: `hgb_log` (Log-Target HGB)	`lr=0.05, max_leaf_nodes=31, min_samples_leaf=50, l2=1.0`	**45.78**	**68.62**	**43.54**	**68.18**	**0.904**	**−26.11**

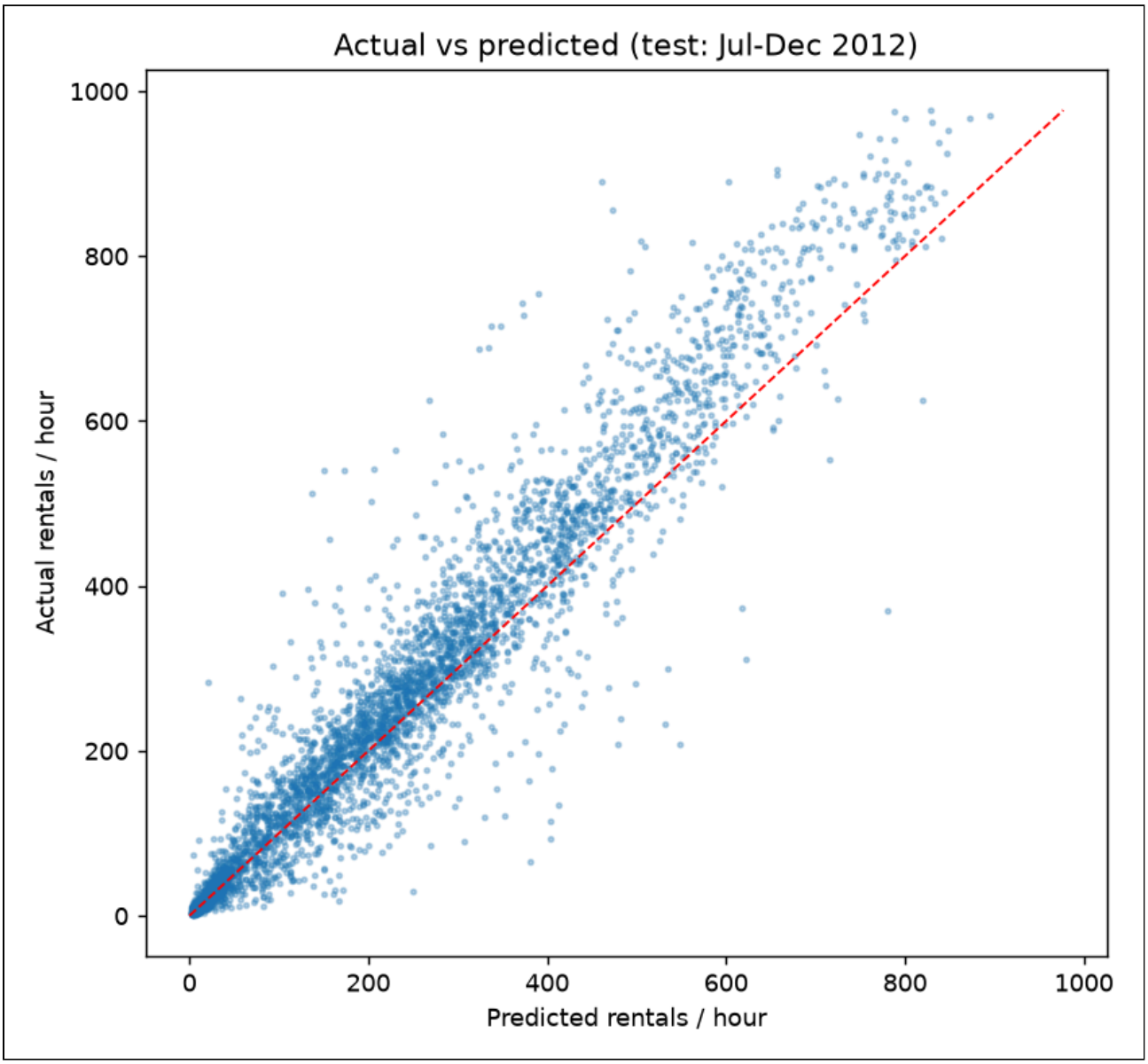


Figure 5. Observed versus predicted hourly bike rentals on the held-out test set (July–December 2012, n = 4,376 hours).

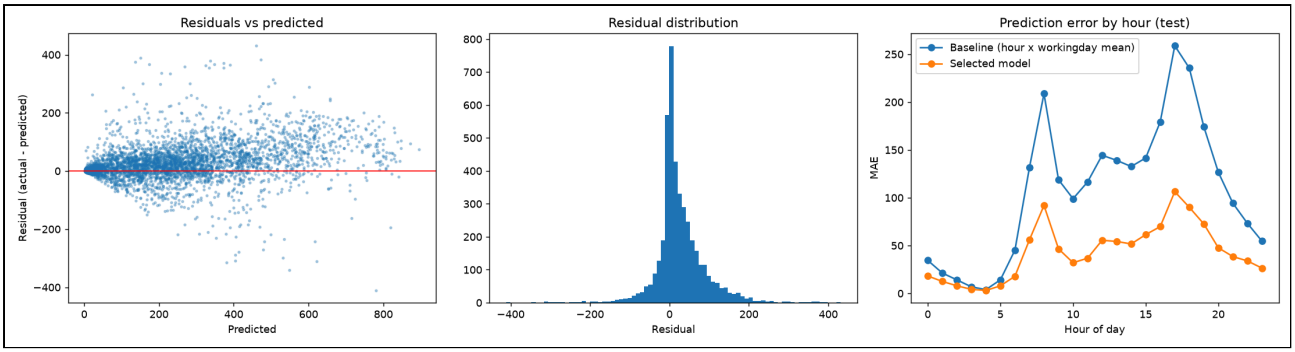


Figure 6. Residual diagnostics on the held-out test period: residuals vs. predicted values (left), residual histogram (center), and hourly MAE comparison against the baseline model (right).

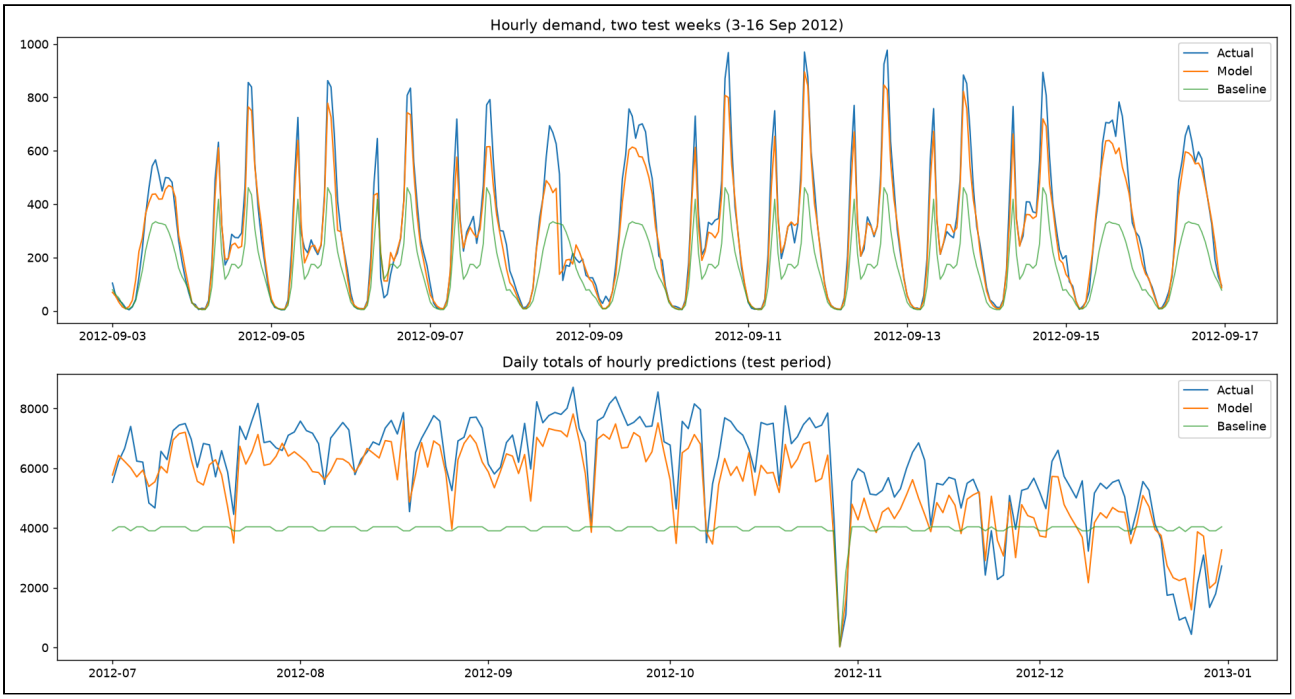


Figure 7. Chronological comparison of actual rentals, gradient boosting predictions, and baseline estimates over a two-week test sample (top) and daily aggregated totals across July–December 2012 (bottom).

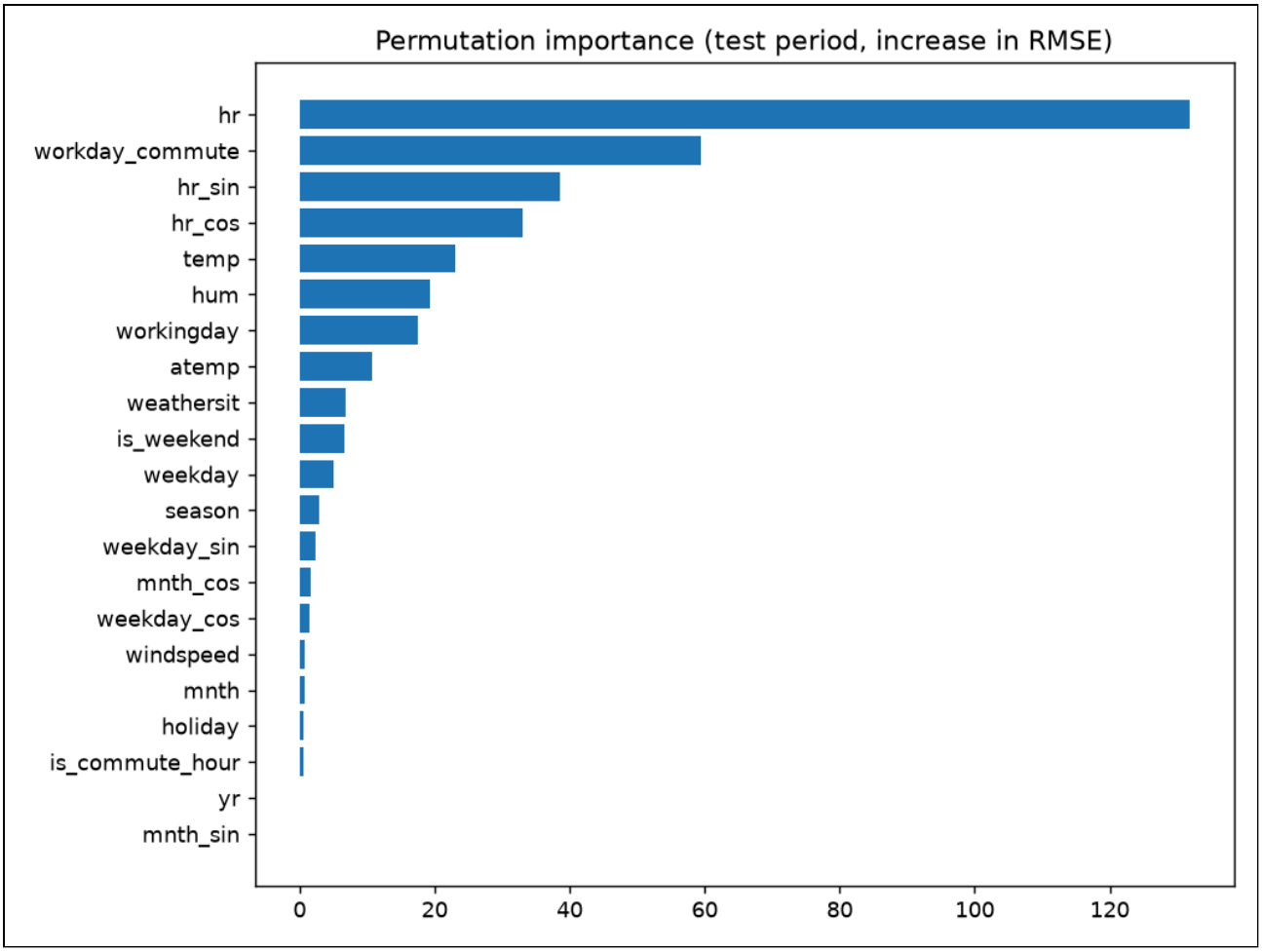


Figure 8. Permutation feature importance measured by increase in out-of-sample test RMSE (July–December 2012).

6. Natural-Language Operations Prototype and Anomaly Detection

6.1 Interactive Prototype Architecture

To make the analytical results accessible to operations managers without programming, I developed a seven-tab Streamlit web application (`app.py`) paired with a deterministic natural-language query engine (`src/query_engine.py`). Rather than relying on an ungrounded generative model that could fabricate statistics, the query engine parses user questions in two stages: (1) an entity extractor (`extract_entities()`) identifies hours, time windows, weather conditions, temperature/humidity/wind values, day types, seasons, and years; and (2) an intent classifier (`detect_intent()`) routes the parsed parameters to 12 deterministic analytical handlers backed by `pandas` and the persisted `demand_model.joblib` pipeline. In addition, the simulator separates statistical point predictions from operational buffer policy by providing a configurable **Operational Safety Buffer** (default `+15%`). Table 4 illustrates representative queries and the exact responses computed by the engine.

Table 4. Representative natural-language operations queries and deterministic outputs computed by `src/query_engine.py`.

Manager Query	Mapped Intent	Computed Empirical Output
"When is bike demand highest on working days?"	`peak_hours`	Peaks at `17:00` (mean `525` rentals/hr, p90 = `782`), followed by `18:00` (`492`/hr, p90 = `731`) and `08:00` (`477`/hr, p90 = `692`).
"Compare demand on working days and non-working days."	`compare_day_types`	Working days average `4,585` rentals/day (bimodal commuter peaks, 84% registered) vs. `4,330` rentals/day on non-working days (midday plateau `12:00–16:00` at `359–373`/hr, 69% registered).
"How does weather affect rental demand?"	`weather_effect`	Within `(hr, workingday, yr, mnth)` cells: Clear is `+8%` (`205`/hr), Mist is `−7%` (`175`/hr), Light rain/snow is `−40%` (`112`/hr), and Heavy rain/snow is `−66%` (`74`/hr).
"Estimate bike rentals at 8 AM in summer on a working day."	`predict`	Point forecast: `578` rentals/hr (80% interval: `461–691`; baseline: `477`). With `+15%` safety buffer: `665` bikes target.
"Compare expected demand during clear weather and light rain."	`predict`	Working day in summer: Clear weather yields `6,003` rentals/day (peak `673`/hr at `17:00`) vs. Light rain `3,764` rentals/day (peak `451`/hr) — a `37%` reduction.
"Which hours should the operations team monitor most closely?"	`monitor_hours`	Ranked by 90th-percentile load: (1) Working day `17:00` (`782`), (2) Working day `18:00` (`731`), (3) Working day `08:00` (`692`), and (4) Non-working day `13:00–15:00` (`590–605`).

6.2 Conditional Anomaly and Event Detection

As an extension to the core deliverables, `src/anomaly.py` identifies unusual system-wide demand shocks in `day.csv` using 8-fold contiguous block cross-fitting. For each day d, out-of-fold expected demand $\hat{y}_d^{\text{OOF}}$ is estimated given calendar and weather predictors, and residuals $r_d = y_d - \hat{y}_d^{\text{OOF}}$ are standardized via the Median Absolute Deviation (MAD):

$$\text{MAD} = \text{median}(|\text{r}_\text{d} - \text{median}(\text{r})|), \quad \text{z}_{\text{robust, d}} = 0.6745 \cdot (\text{r}_\text{d} - \text{median}(\text{r})) \,/\, \text{MAD} \tag{11}$$

Flagging dates with $|z_{\text{robust, }d}| \geq 3.0$ or percentage shortfall $\leq -60\%$ isolates 12 major operational anomalies (Table 5), including the Hurricane Sandy municipal shutdown on 29–30 October 2012 (`−99.1%` and `−61.6%`), the January 2011 winter blizzard (`−82.2%`), unmodeled Christmas Eve demand drops (`2012-12-24`, coded as a regular working day in the UCI dataset but exhibiting a `−70.5%` shortfall), and the 2012 National Cherry Blossom Festival / St. Patrick's Day surge (`+258.2%`).

Table 5. Top positive and negative daily demand anomalies identified via cross-fitted residual screening.

Date	Observed	Expected	Residual (% Diff.)	Robust z	Associated Historical Event in Washington, D.C.
`2012-10-29`	22	2,451	−2,429 (−99.1%)	−3.09	Hurricane Sandy landfall (system suspended after 00:00; 23 missing hours)

Date	Observed	Expected	Residual (% Diff.)	Robust z	Associated Historical Event in Washington, D.C.
2011-01-27	431	2,418	−1,987 (−82.2%)	−2.53	January 2011 Mid-Atlantic winter snowstorm and icy road conditions
2012-12-24	920	3,115	−2,195 (−70.5%)	−2.80	Christmas Eve (coded as `workingday = 1`, but commuters took holiday leave)
2012-10-30	1,096	2,855	−1,759 (−61.6%)	−2.24	Hurricane Sandy Day 2 recovery and partial afternoon service resumption
2012-03-17	7,836	2,187	+5,649 (+258.2%)	+5.18	St. Patrick's Day and National Cherry Blossom Festival warm Saturday surge
2012-03-23	8,362	5,647	+2,715 (+48.1%)	+3.42	Peak Cherry Blossom bloom warm Friday (25°C)

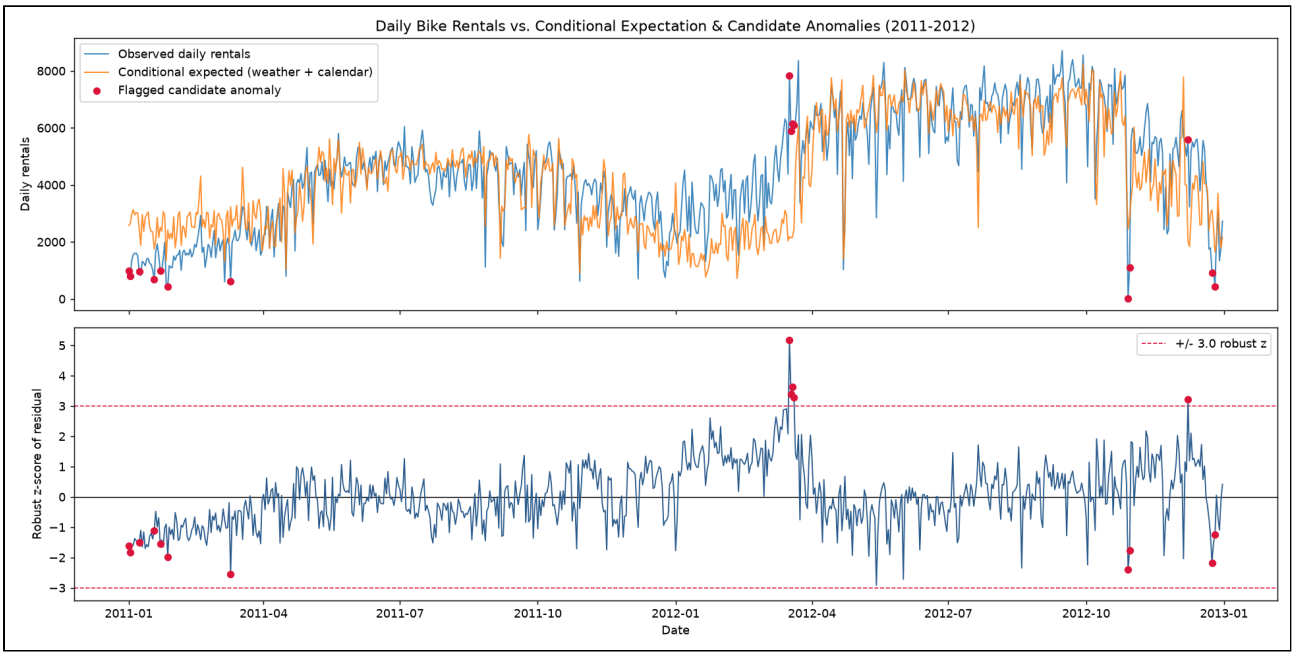


Figure 9. Observed daily rentals versus cross-fitted conditional expectation (top) and robust MAD residual z-scores highlighting major weather and holiday anomalies (bottom).

7. Project Codebase Structure and Reproducibility

The project is packaged as a self-contained Python repository verified by 24 automated unit, integration, and headless Streamlit tests in `tests/test_project.py`. Table 6 lists the core modules and their responsibilities.

Table 6. Summary of repository modules and verification scripts.

Module / File Path	Responsibility
`src/config.py`	Central directory paths, commuting-window definitions, feature lists, and UCI category mappings.
`src/data_validation.py`	Executes 26 automated quality checks on `hour.csv` and `day.csv` and writes `reports/data_quality_report.md`.

Module / File Path	Responsibility
`src/features.py`	Deterministic feature engineering (Fourier harmonics, physical units, rush-hour interactions) and `assert_no_leakage()`.
`src/hypothesis.py`	Welch's *t*-test, Mann-Whitney *U*, bootstrap CIs, and Newey-West HAC regression (`reports/hypothesis_results.md`).
`src/train_model.py` & `src/predict.py`	Expanding-window CV, model training, residual quantile intervals, and non-negative scenario forecasting.
`src/query_engine.py` & `app.py`	Deterministic natural-language intent/entity parser and 7-tab interactive Streamlit operations dashboard.
`src/anomaly.py`	8-fold block cross-fitted daily residual anomaly detector (`outputs/anomalies.csv`).
`tests/test_project.py`	24 automated `pytest` tests covering data quality, leakage guards, non-negative predictions, and UI rendering.

To reproduce all analytical outputs, run the test suite, or launch the interactive prototype locally, execute the following commands from the project root:

```
.\.venv\Scripts\python.exe -m src.data_validation
.\.venv\Scripts\python.exe -m src.features
.\.venv\Scripts\python.exe -m src.hypothesis
.\.venv\Scripts\python.exe -m src.train_model
.\.venv\Scripts\python.exe -m src.anomaly
.\.venv\Scripts\python.exe -m pytest -v
.\.venv\Scripts\streamlit.exe run app.py
```